Misconfigured by Default: The Prevalence, Validity, and Tool Coverage of Kubernetes Misconfigurations in the Wild

Girish Mallya[1*]

[1*]Independent Researcher, Bellevue, WA, USA.

Corresponding author(s). E-mail(s): girishm60@gmail.com;

Abstract

Kubernetes manifests are the de facto interface to production infrastructure, with permissive defaults: a workload specifying nothing runs with an automounted token, no resource limits, and no sandboxing profile. We measure how 151 Helm charts and 60 public GitHub repositories (16,243 documents, 2,811 workloads) configure these knobs, using a 25-rule engine.

We validated the engine on a 457-finding stratified sample: AI-generated pre-labels were author-checked — the 92 protocol-sensitive rows individually adjudicated, a blinded 30-row subset labeled blind (author–AI agreement: $\kappa = 0.526$ [0.17, 0.84]) — the rest retain accepted pre-labels, 16 later corrected by a blinded re-labeling audit. Population-weighted precision is 0.676 [0.60, 0.81] (cluster bootstrap by source): ≈18,600 [16,500, 22,100] of 27,460 raw findings are estimated genuine. Precision is rule-dependent: unambiguous absence checks (seccomp, privilege escalation, root prevention) are essentially exact, while context-sensitive rules (committed secrets, host namespaces, probes) are mostly benign-by-design cases.

The defaults dominate: tokens automount in 88.6% of production GitHub workloads, CPU limits — a contested hardening practice — are missing in 75.3%; 1.4% run a privileged container. Helm charts beat raw manifests on most practices, but by only 1.3–1.7× in workload-level prevalence.

We also audit the auditors: on identical input, four scanners cover at most 16 of the 25 practices, diverging up to 1.4× on the same practice through counting semantics. Scored against our reference labels (Helm arm, n = 126), they effectively detect 51–72% of the engine's reference-positive findings and flag 84–95% of reference-benign findings where a check exists, under our contextual protocol. static analysis tools; measurement validity

Keywords: Kubernetes, configuration errors, empirical study, misconfiguration,

0.1 1. Introduction

Kubernetes is now the default platform for production containerized workloads, and a Kubernetes workload is typically described by a YAML manifest that its author wrote once and rarely revisits. The platform's configuration surface is large, and its defaults were chosen for convenience rather than safety. A container that says nothing about security runs as whatever user its image specifies, with a service-account token mounted into its filesystem, no CPU or memory limits, no seccomp profile, and privilege escalation allowed. Each of these is a documented hardening gap; in aggregate, they define the actual security posture of the cloud-native ecosystem.

Two questions follow. First, *descriptive*: how are real manifests actually configured — which hardening practices are routine, which are rare, and how much of the apparent risk is an artifact of counting test fixtures and examples as if they were production code? Second, *instrumental*: the ecosystem's answer to this problem is a family of static auditing tools, but those tools are themselves rarely audited. If four widely used scanners are run on identical input, do they cover the same practices, and do their counts agree?

Both questions are measurement questions, and both are easy to answer badly. Manifest corpora scraped from public sources mix production code with tutorials, test fixtures, and rendered templates; naive counting silently grades homework as if it were infrastructure. Scanner comparisons, meanwhile, tend to report raw finding counts without establishing whether any tool's findings are *correct* — and our own validation shows that for several common rules, the majority of flagged findings are benign in context.

This paper makes four contributions:

1. **A two-arm prevalence measurement** of 25 misconfiguration practices across 16,243 documents (2,811 workloads), with source tagging that separates production manifests from examples and fixtures, cluster-level prevalence that corrects for duplicated content, and a Helm-vs-GitHub comparison that corrects a best-chart bias in the obvious sampling strategy (Section 4.1, 4.2).
2. **A validated rule engine.** Every prevalence number is paired with a per-rule precision estimate from a 457-finding stratified sample, judged under a written labeling protocol whose evolution (three versions), blinded author–AI agreement check, and adjudication amendments are documented and released. Precision, not raw counts, is the paper's unit of evidence (Section 4.3).
3. **A revised taxonomy** of Kubernetes misconfiguration that distinguishes misconfigurations proper from hardening gaps and manifest rot, and classifies false positives into reusable context classes (infrastructure agents, credential-free placeholders, standard operator patterns) (Section 4.3).
4. **An audit of four production scanners on identical input**, showing that coverage tops out at ˜16 of 25 practices, that agreement is high only where misconfiguration is explicit, and that headline count divergences (up to ˜1.4× on the same practice) are explained by counting semantics — pod- vs container-level units and merged checks; and that, scored against the reference labels, the tools' real deficits

— for the versions and default configurations evaluated here — are coverage and context-sensitivity, not detection on the practices they do check (Section 4.4).

The headline result is that in this corpus the risk is concentrated in *defaults*, not in explicit misbehavior. Privileged containers appear in 1.4% of production workloads; automounted service-account tokens appear in 88.6%. The pattern suggests security effort — human and tooling alike — is calibrated to the rare, explicit cases and largely blind to the common, default ones.

0.2 2. Background and research questions

A Kubernetes workload manifest declares, among other things, a pod's security context (user, capabilities, seccomp and AppArmor profiles, privilege escalation), its containers' resource requests and limits, image references (tag and digest pinning), health probes, service-account binding, and — in adjacent objects — RBAC roles and Secrets. Nearly every field is optional, and the platform supplies a permissive default when a field is absent. Misconfiguration, for this study, is a manifest state that a documented hardening practice (the CIS Kubernetes Benchmark, the NSA/CISA hardening guide, and the scanners' own rule catalogues) identifies as a gap. Section 3.2 defines the 25 rules operationally; Section 3.4 describes how we establish that the rules measure what they name.

We ask four research questions:

- **RQ1 (Prevalence).** How common is each class of misconfiguration in real-world manifests, once production code is separated from examples and fixtures and duplicated content is accounted for?
- **RQ2 (Validity and taxonomy).** With what precision does a rule-based engine detect each practice — i.e., which of the 25 measurements can be taken at face value and which require contextual judgment — and what taxonomy organizes the true and false positives?
- **RQ3 (Tool coverage).** On identical input, how much of the observed taxonomy do the four widely used scanners cover, and do their counts agree? Where they diverge, is the divergence detection or bookkeeping?
- **RQ4 (Curation effect).** Do rendered Helm charts — the ecosystem's curated, templated distribution channel — differ from manifests committed directly to Git repositories, and by how much once sampling bias is handled?

0.3 3. Method

0.3.1 3.1 Corpus construction

The corpus has two arms, collected in October 2026 from public sources only.

Helm arm. We rendered the 151 most-starred charts on Artifact Hub — the registry's package search (kind = Helm), collected 2026-10-21, with every chart's name and version pinned in `results/helm_render_log.json` — with default values (`helm template`), yielding 2,616 documents and 402 workloads. Default values are what an operator gets when they install a chart without tuning it — the correct object of

study for a question about defaults — but a top-charts sample over-represents well-maintained projects; Section 4.2 quantifies and corrects for this *best-chart bias* rather than ignoring it.

GitHub arm. We mined 60 public repositories — a seed set of named projects (9 initial seeds, 12 more added at scale-up) plus repositories discovered through the GitHub repository search API (four manifest-oriented queries, results sorted by stars, repository size restricted to 200 KB–400 MB, discovery capped at 40 repositories) — yielding 13,627 documents and 2,409 workloads. Snapshots are pinned by commit SHA (`results/github_commit_shas.json`). Every file was source-tagged as *production*, *example*, or *fixture* by path and repository role, so prevalence can be reported for production manifests alone; an addendum to the labeling protocol (Section 3.4) resolves tag/content conflicts in favor of content (e.g., a `testdata/` tree inside a production-tagged repository counts as fixture).

The corpus itself is not redistributed (third-party manifests carry mixed licenses); the collection scripts and per-source logs are the corpus manifest, and running the collectors rebuilds it.

0.3.2 3.2 The rule engine

We implemented a 25-rule engine over parsed YAML (Deployments, StatefulSets, DaemonSets, Jobs, CronJobs, Pods, and the RBAC/Secret objects adjacent to them). Rules span six families: **images** (untagged, `:latest`, no digest pinning), **resources** (missing CPU/memory limits, missing requests), **reliability** (missing liveness/readiness probes), **security context** (privileged, root not prevented, explicit root, privilege escalation not disabled, dangerous or any added capabilities, missing seccomp), **host access** (host network/PID/IPC, automounted SA token), **RBAC and secrets** (wildcard verbs/resources, cluster-admin binding, committed Secret objects, literal secrets in environment variables), and **API currency** (deprecated APIs — reported as manifest rot, not misconfiguration; see Section 4.3). Engine v3 incorporates three correctness fixes over v2 that matter for the results: the SA-token rule joins the ServiceAccount object's own automount flag; init containers are excluded from probe rules (a third of v2's Helm probe findings were on init containers); and `seccompProfile.type=Unconfined` counts as no profile. Engine v3 reports 27,460 findings over the full corpus. The full rule definitions ship with the replication package.

0.3.3 3.3 Tool audit setup (RQ3)

We ran kube-linter, kubeaudit, Polaris, and Checkov — four widely used open-source manifest auditors — on the *identical* Helm-arm input (402 workloads, rendered with default values), each with its default configuration. Default configuration is the treatment of interest: it is what a new adopter runs, and, as Section 4.4 shows, several relevant checks are not in it. We mapped each tool's checks to our 25 rules by hand from the tools' output on this input, recording partial coverage (a check that covers a rule with narrower semantics) as one half in coverage totals. Counts were reconciled rule by rule; where a tool merges two of our rules into one check (e.g., untagged and `:latest`), we report the merged figure rather than forcing an attribution. Separately, we score each tool against the reference labels: the 126 Helm-arm rows of the

validation sample (Section 3.4) carry protocol-based reference labels (AI-annotated, author-adjudicated), so for every sampled finding we can ask whether the tool reported the corresponding practice on that object — yielding detection rates on reference-positive findings and flagging rates on reference-benign findings, per tool (full method and per-row data in `results/tool_accuracy.md`).

0.3.4 3.4 Validation: labeling protocol, reliability, adjudication

Prevalence without precision is numerology, so we validated the engine on a 457-finding stratified sample: all findings of the two rarest high-severity rules, stratified draws from the rest, and ˜10% hidden duplicate rows. A sample row is one finding with an *evidence packet* (the flagged object, its security context, and its source context).

Labels (TP / FP / unsure) were assigned under a written protocol. The protocol's history is part of the method and is reported, not hidden: an initial *definitional* standard (v2: a finding is TP iff the manifest exhibits the property the rule names, in context) was harmonized with an earlier *exploitability* draft (v1) into a governing protocol (v3) that keeps the definitional core and adds explicit contextual amendments — production scoping for resource/probe rules, a two-tier treatment of digest pinning, an escalation-path test for root/privilege-escalation rules, and a credential-material test for committed secrets. AI pre-labels produced under v2 were treated as an *initial annotation pass*, never as ground truth.

Blinded author–AI agreement. The author labeled a blinded 30-row subset (spanning all 25 rules) before seeing any pre-labels. Agreement was 24/30 (80%), Cohen's $\kappa = 0.526$ [bootstrap 95% CI 0.17, 0.84] against the v2-standard pre-labels. This is agreement between the author and AI-generated labels, not inter-rater reliability between two humans; and because the pre-labels follow v2 while the author's labels follow the emerging v3, κ also absorbs the protocol-version difference. The κ figure is the only agreement measurement in this study; no independent second labeling was performed (a same-author blind re-labeling audit of the retained labels follows in the verification paragraph below). We report this number as measured, with its decomposition, because the decomposition is the finding: four of six disagreements were resource/probe rows where the governing v3 production-scoping amendment sides with the author against the v2 definitional labels — a standards difference, not rater noise — and the remaining two were genuine judgment calls (a CNI agent's dangerous capabilities; an operator's wildcard RBAC confined to its own API groups), which were adjudicated into two further *rule-level* amendments (an infrastructure-class test requiring a drop-ALL baseline plus function-required capabilities, and an own-API-group scoping test for wildcard RBAC), adopted before the verification pass and now part of the protocol (`results/labeling_protocol_v3_addendum.md`). Intra-rater consistency, measured by the hidden duplicates, was 44/44 groups.

Verification. The author then ran a verification pass over the full sample: the 92 *divergence rows* — those whose labels can differ between the v1 and v2 standards — were adjudicated first, and the author accepted the v3 recommendations on all 92 without override on 2026-10-27; the remaining rows were reviewed in aggregate (blank = accept, per protocol). The provenance of the final labels is therefore mixed, and we state it exactly: of the 451 sample rows that persist in engine v3, **111 (25%)**

carry labels the author produced or individually adjudicated — the 30 blind-labeled rows and 87 adjudicated divergence rows, with 6 rows in both sets — while **340 (75%) retain the AI pre-labels** the author accepted during the pass. The validation dataset is thus AI-annotated and author-adjudicated, not independently human-labeled; what that does and does not establish is taken up in Section 6.

Stability audit of the retained labels. As a check on the 340 retained labels, the author blindly re-labeled a stratified subsample of 100 findings drawn from them (seed 20261103; drawn per rule in proportion to the retained population, and spanning 22 of the 25 rules — the other three rules have no retained rows), without access to the original labels or their rationales. The subsample stands for 108 of the 340 rows. Of the 94 findings the author decided (6 unsure, labels kept), 16 changed label — 17.0% [95% CI 10.8, 25.9] — 10 from TP to FP and 6 from FP to TP, each standing for exactly one sample row; the disagreements concentrate in the context-sensitive rules (service-account automount, run-as-root, literal secrets, wildcard RBAC), while the absence checks were unchanged. The author adopted the audit labels as finals for all 16 rows (adjudication by rule, 2026-11-04), so 324 of the 340 rows carry the accepted pre-label and 16 an audit-corrected label; all headline figures are computed on the corrected labels. The audit is a same-author stability check — it measures how reproducibly one rater applies the frozen protocol to the retained labels, not agreement between independent raters.

Rows whose flagged property cannot be observed in the source (7, e.g. images injected by kustomize overlays) are excluded from precision denominators with the count and reason disclosed.

0.3.5 3.5 Analysis

For each rule we report precision = TP / (TP + FP) over decided rows with Wilson 95% confidence intervals, *adjusted prevalence* = precision × population findings, and workload-level prevalence (share of workloads with ≥1 finding) per corpus arm and source tag. Because the validation sample was drawn under engine v2 and the population is engine v3, we reconciled every sampled row against v3 findings on (arm, source, kind, name, container, rule): 451 of 457 rows still exist in v3; the headline figures are computed on that matched frame, and a distinct-finding sensitivity check (one vote per unique key, collapsing the intentional duplicates) is reported alongside. Two aggregates of the per-rule precisions answer different questions and are kept distinct throughout: the rate across sampled rows (0.624) describes the *sample*, in which rare rules are deliberately over-represented because all findings of the rarest rules were taken; the population-weighted aggregate — each rule's precision weighted by its v3 population, equivalently the adjusted finding count over all findings — is **0.676** (18,556 / 27,460) and is the figure that characterizes the corpus as a whole. Propagating the per-rule binomial sampling variances, the adjusted intervals are cluster-bootstrapped by source (resampling repositories and charts, 2,000 replicates; `results/cluster_bootstrap.md`): the adjusted total's 95% interval is [16,500, 22,118] and the weighted precision's is [0.601, 0.805]. Row-level intervals that treat findings as independent are materially narrower and are not used, since findings within a source are correlated (52.9% of v3 findings repeat a seen key). The intervals are also

insensitive to how a replicate treats a rule with no decided rows in the resample — an event in only 42 of 50,000 rule-replicates: full-sample, pooled-precision, and drop-and-renormalize fallbacks all yield [0.601, 0.805] to within 0.001. Every adjusted prevalence figure in Section 4 is built per rule and is identical under either aggregate. As a standards sensitivity check, the same sample scored under the superseded v2 definitional standard (the pre-labels) gives ≈0.73; the gap to 0.624 is the v3 amendments — production scoping and the context classes of Section 4.3 — reclassifying findings that the purely definitional standard counted as genuine. Cluster-level prevalence (share of repositories/charts with ≥1 finding) addresses the non-independence of findings within a source: 52.9% of v3 GitHub findings repeat an already-seen (kind, name, container, rule) key, so document-level rates alone would overstate how widespread each practice is — and, as Section 4.1 shows, the practices are widespread anyway.

0.4 4. Results

0.4.1 4.1 RQ1 — Prevalence: the defaults are the finding

Table 1 reports workload-level prevalence for the headline practices, per arm, with the GitHub arm restricted to production-tagged manifests. Three facts stand out.

Default-insecure settings are near-universal. In production GitHub manifests, service-account tokens are automounted in **88.6%** of workloads and CPU limits are missing in **75.3%**; in rendered Helm charts the figures are 85.1% and 85.6%, with **97.5%** of Helm workloads running at least one image that is not digest-pinned (65.3% for production GitHub). Restricting to production code barely moves the numbers (corpus-wide GitHub SA automount is 96.2%): the risky defaults live in shipped manifests, not in test files.

Explicit dangers are rare. Privileged containers appear in **1.4%** of production GitHub workloads (5.5% in Helm charts, where node-level agents are common), and cluster-level prevalence tells the same story from the other side: SA automount appears in **94%** of repositories and **90%** of charts, missing seccomp in 94%/72%, unpinned digests in 91%/97% — but privileged containers in only 22% of repositories and 8% of charts. The ecosystem has largely learned not to do the obviously dangerous thing, and has largely not learned to turn off the defaults.

Table 1. Workload-level prevalence, engine v3 (share of workloads with ≥1 finding).

Practice	Helm charts	GitHub (production)	GitHub (all)
Image not digest-pinned	97.5%	65.3%	—
SA token automounted	85.1%	88.6%	96.2%
No CPU limit	85.6%	75.3%	79.0%
Privileged container	5.5%	1.4%	—

Full per-rule tables for all 25 rules, both arms and all source tags, are in the replication package (`results/stats_tagged_v3.json`).

0.4.2 4.2 RQ4 — Curation helps, less than it looks

Rendered Helm charts are better hardened than raw GitHub manifests, but the gap is **1.3–1.7**×, not the 2–3× suggested by comparing the most popular charts against unfiltered repository mining. The mechanism is sampling: a top-151-charts frame overweights flagship projects with dedicated maintainers. Even taking the favorable frame at face value, the curated channel's advantage is concentrated in the explicit-danger rules; on default-insecure practices (automount, limits, digest pinning) charts are statistically indistinguishable from — and on CPU limits and digest pinning worse than — production GitHub code. Curation polishes the visible edge of the configuration surface; it does not change the defaults underneath.

0.4.3 4.3 RQ2 — Precision and taxonomy: which numbers can be believed

Overall precision on the reconciled, author-adjudicated sample is **0.624 [0.519, 0.705]** (275 TP / 166 FP over decided rows; cluster-bootstrap interval by source), adjusting the 27,460 raw findings to an estimated **≈18,600 genuine misconfigurations** (95% CI [16,500, 22,100] — a model-based extrapolation of the per-rule estimates, not a count of individually labeled findings). A distinct-finding sensitivity analysis gives 0.613 — the intentional duplicate rows do not move the result. Population-weighted, overall precision is 0.676 (Section 3.5); the per-rule adjustments that follow are the same under either aggregate. But the aggregate conceals the paper's most useful variation: precision is a property of the *rule*, and it splits the taxonomy into rules that can be read at face value and rules that cannot.

Face-value rules (precision ≈ 1.0). Missing seccomp profile, privilege escalation not disabled, root not prevented, and `:latest` images are essentially never benign in context (all 1.000 on their samples); dangerous capabilities (0.824, under the addendum's infrastructure test), wildcard RBAC (0.71–0.80, under the own-group test), and SA-token automount (0.643) follow. For these, scanner output *is* prevalence, up to sampling error.

Context-dominated rules (precision ≤ 0.4). Committed Secret objects are the extreme: **0.125**. Under the credential-material test, most committed Secrets are empty, operator-populated, or documented placeholders; the genuine exposure is a credential-bearing minority that no content-blind rule can separate from the noise. Host network (0.312) and host PID (0.214) findings are mostly node infrastructure — CNI/CSI agents and node exporters whose function requires the access. Missing liveness probes (0.214) and the resource rules (0.38–0.43) are dominated by production scoping: examples, fixtures, and one-shot Jobs that a context-free count grades as production gaps. Literal secrets in env values (0.267) are mostly test fixtures and template placeholders. Added capabilities (0.176) are mostly least-privilege patterns (default-set additions, or `NET_BIND_SERVICE` over a drop-ALL baseline).

Table 2. Per-rule precision (reference labels, reconciled to engine v3; Wilson 95% CIs). Selected rules; full table in `results/precision_provisional.md`.

Rule	Precision	95% CI	Reading
SEC_NO_SECCOMP	1.000	[0.82, 1.00]	face value
SEC_APE_NOT_DISABLED	1.000	[0.80, 1.00]	face value
SEC_ROOT_NOT_PREVENTED	1.000	[0.78, 1.00]	face value
IMG_LATEST	1.000	[0.80, 1.00]	face value
SEC_DANGEROUS_CAPABILITY	0.824	[0.59, 0.94]	face value
RBAC_WILDCARD_VERBS	0.800	[0.55, 0.93]	mostly
RBAC_WILDCARD_RESOURCES	0.706	[0.47, 0.87]	mostly
SA_TOKEN_AUTOMOUNTED	0.643	[0.39, 0.84]	mostly
SEC_PRIVILEGED	0.595	[0.48, 0.70]	mixed
REL_NO_READINESS_PROBE	0.538	[0.29, 0.77]	context-heavy
RES_NO_CPU_LIMIT	0.429	[0.21, 0.67]	context-heavy
RES_NO_REQUESTS	0.412	[0.22, 0.64]	context-heavy
RES_NO_MEM_LIMIT	0.375	[0.18, 0.61]	context-heavy
HOST_NETWORK	0.312	[0.14, 0.56]	context-dominated
SECRET_LITERAL_ENV	0.267	[0.11, 0.52]	context-dominated
HOST_PID / REL_NO_LIVENESS_PROBE	0.214	[0.08, 0.48]	context-dominated
SEC_CAPABILITY_ADDED	0.176	[0.06, 0.41]	context-dominated
SECRET_COMMITTED	0.125	[0.03, 0.36]	context-dominated

Taxonomy v2. The validation exercise reorganizes the 25 rules into a three-tier taxonomy, and the false positives into five reusable context classes:

- **Tier 1 — Misconfigurations.** States that are wrong in essentially all contexts (the face-value rules above, plus wildcard RBAC and cluster-admin binding at 0.71–0.90 with the own-group exception below).
- **Tier 2 — Hardening gaps.** States that are genuine but lesser gaps, reported separately and never folded into misconfiguration precision: digest-unpinned images *with* an explicit version tag (7 of 14 sampled no-digest rows), and the context-heavy resource/probe rules when they occur in production code.
- **Tier 3 — Manifest rot.** Deprecated API versions: real, mechanically certain, and not a security misconfiguration; reported as their own line.

False positives concentrate in five context classes, each of which became an explicit protocol test: **infra-need** (node agents whose function requires the access, gated on a drop-ALL baseline for capabilities), **placeholder** (Secrets and env values without credential material), **fixture** (test/example content, decided by content over source tag), **standard operator pattern** (wildcard RBAC confined to the operator's own API groups; least-privilege capability additions), and **not-exploitable** (root/APE findings with no escalation path: non-root enforced and no added capabilities). A scanner — or a study — that implements these five distinctions recovers most of the precision the naive rules lose.

0.4.4 4.4 RQ3 — Auditing the auditors

On identical Helm-arm input with default configurations, approximate coverage of the 25 practices is: Checkov ˜16, Polaris ˜13, kube-linter ˜11, kubeaudit ˜11 (partial coverage counted as one half). Three results matter more than the ranking.

Nobody covers the long tail, and one gap is total. Digest pinning — the single most prevalent practice in the corpus — is checked by exactly one tool (Checkov CKV_K8S_43). Wildcard RBAC has no *default-enabled* check that catches it broadly: Checkov's CKV_K8S_49 flags 29 of the 155 wildcard findings on this input, and kube-linter ships wildcard checks that are off by default. And **no tool, in the configuration run, inspects committed Secret contents** (Checkov ships a separate secrets framework, outside the Kubernetes checks we ran): all four flag (or ignore) Secret objects without distinguishing credential material from placeholders, which Section 4.3 shows is a 7:1 noise-to-signal distinction. The most sensitive objects in the corpus are the ones the tooling understands least.

Agreement exists only where danger is explicit. On privileged containers, the four tools report 26–27 findings against our 27 — near-perfect consensus. On missing CPU limits, the same input yields **343 / 411 / 416 / 497** findings across the tools against 502 container-level cases in our engine. The spread is bookkeeping, not detection: kubeaudit counts at pod level and merges "no limits at all" (421) with "limits without CPU" (76); kube-linter merges limits with requests (416); Checkov counts a narrower property (343). A consumer comparing tools — or tracking one tool over time — sees a ˜1.4× swing that has nothing to do with the underlying manifests.

Accuracy against the reference labels. Coverage and counts say what a tool *can* check and how much it *reports*; the labeled sample lets us ask whether it is *right*. Where a tool has a check for a sampled practice, it detects reference-positive findings reliably — kubeaudit 0.98, Polaris 0.93, kube-linter 0.90 — with Checkov the outlier at 0.77 (its run-as-non-root check systematically misses containers that run as root *explicitly*: 6 of its 14 misses). Counting practices with no check as missed, effective detection of genuine misconfigurations is 0.51–0.72, and the ranking inverts the raw-count intuition: Polaris leads (0.72) on the strength of coverage, not per-check detection. The symmetric result is the more damning one: the same checks flag **84–95% of the findings verification judged benign (reference-benign findings)**. The taxonomy's context classes — infrastructure agents, standard operator patterns, least-privilege capability sets — fool the production tools at about the same rate they fooled our unvalidated rules. The tools' deficit on real manifests is not detection; it is coverage and context. These are sample-based estimates from the 126 Helm-arm validation rows, drawn from the engine's own findings: the flagging figures are shares of sampled findings that verification judged benign, not false-positive rates over all negative configurations, which a findings-only sample cannot measure. The benchmark for "benign" is likewise ours: the tools implement the definitional reading of each practice and do not claim the v3 context classes, so the flagging rates measure the distance between definitional checking and context-aware judgment — the quantity a user who trusts the output experiences — rather than vendor error against the tools' own specifications.

Tool	TP detection (check exists)	TP detection (effective)	FP-flagging (check exists)
kube-linter	0.90 [0.77, 0.96]	0.51	0.92
kubeaudit	0.98 [0.87, 1.00]	0.55	0.95
Polaris	0.93 [0.83, 0.97]	0.72	0.88
Checkov	0.77 [0.66, 0.86]	0.66	0.84

Reference labels: 73 reference-positive and 50 reference-benign Helm-arm sample rows; rates over rows where a mapped check exists, except the effective column, which counts no-check practices as missed. Method, joins, and caveats: `results/tool_accuracy.md`.

Defaults decide what gets found. kube-linter's probe checks validate probe *port correctness*, not probe existence; its SA-token and wildcard checks are absent or opt-in; kubeaudit's SA check covers only the default service account. The pattern is consistent: each tool's default configuration is tuned to the explicit-danger tier of the taxonomy, and the default-insecure tier — where Sections 4.1 and 4.3 show most genuine misconfiguration lives — is covered partially, optionally, or not at all.

0.5 5. Discussion

The risk is the default, and the default is a design outcome. The practices with the highest prevalence — automounted tokens, absent limits, unpinned images, absent seccomp — are precisely those where doing nothing produces the finding. Manifest authors are not choosing insecurity; they are accepting a platform posture. A scope note on what the findings are: most are *exposures and hardening gaps*, not demonstrated vulnerabilities — an automounted token's impact depends on the permissions bound to its service account and on cluster context, which a manifest alone does not determine. The taxonomy's tiers (Section 4.3) keep that distinction in view; the prevalence claims are about configuration posture, not exploitability. Interventions aimed at author behavior (linters in editors, review checklists) fight the last war: the durable fix is at the platform and distribution layer — admission-time defaults (Pod Security Standards, policy engines with the checks *enabled*), chart templates that ship hardened security contexts, and image pipelines that pin digests at build time. Our curation result (Section 4.2) is the cautionary evidence: even the ecosystem's most maintained artifacts only partially overcome the defaults.

Scanner output needs a validity label. Two scanners can disagree by 1.4× on the same manifests without either being "wrong" — they are counting different units and different properties under the same English name. We suggest the field adopt three reporting norms: state the counting unit (pod vs container), state whether merged checks are decomposed, and state per-check precision on a public validation corpus, as this study does for its own engine. A count without those three qualifiers is not comparable across tools or across time.

The five context classes are a checklist for tool authors. Most of the precision our naive rules lost is recoverable with distinctions that are mechanical once named: join the ServiceAccount object before grading automount; require credential

material before calling a Secret committed; treat a drop-ALL baseline plus function-required capabilities on a node agent as infrastructure, not exposure; scope wildcard RBAC by API-group ownership; and never grade fixtures as production. Several of these are one-join or one-test changes. The tools' roadmaps, not their detection cores, are the bottleneck.

For practitioners, the actionable reading of Tables 1 and 2 is short: the face-value rules are a reliable triage list (fix seccomp, privilege escalation, root prevention, and `:latest` first — the findings are real), SA tokens and resource limits are the largest genuine exposure by volume, and committed-Secret alerts should be routed to content inspection rather than counted.

0.6 6. Threats to validity

Construct validity rests on the labeling protocol. We mitigate by *disclosing* rather than concealing its evolution: three protocol versions, a blinded author–AI agreement subset ($\kappa = 0.526$ against the v2-standard labels, with the disagreement decomposition showing 4 of 6 tracing to the v2→v3 standards change itself), adjudicated rule-level amendments, and author adjudication of the labels (Section 3.4 gives the row-level provenance). The ground truth is thus single-author and AI-assisted: every reference label rests on one author's adjudication, and no independent labeling pass checks it. The blinded re-labeling audit of Section 3.4 mitigates this only partly: it caught and corrected 16 retained labels, but a same-author audit measures the stability of the protocol's application, not its agreement with an independent rater. We regard the absence of independent labeling as the validation's main limitation. Readers who prefer the purely definitional standard can recompute precision under it from the released sample (the v2 pre-labels are preserved); the prevalence *rankings* are unchanged under either standard, though several per-rule precisions move, as Section 4.3 reports. The κ figure is below the conventional 0.7 benchmark and we do not claim otherwise; it measures agreement with labels produced under a superseded standard, and the verification pass — not the κ subset — is the ground truth of record.

Internal validity. Engine v3's three semantic fixes (SA-object join, init containers in probe rules, Unconfined seccomp) each moved findings materially; residual engine bugs would bias prevalence and are mitigated by the validation sample itself — a systematically wrong rule shows up as low precision, and none of the face-value rules do. For RQ3, check-to-rule mapping was manual; we mitigated by reconciling counts rule by rule and reporting merged checks as merged rather than forcing attribution.

External validity. The GitHub arm over-represents public, open-source practice and under-represents private enterprise manifests, which plausibly differ in both directions (more policy enforcement, more legacy). The Helm arm's best-chart bias is measured in Section 4.2 and cuts toward *understating* ecosystem-wide prevalence. All data is from October 2026; defaults and tooling evolve, and the replication package is designed for re-collection.

Reliability. Intra-rater consistency was 44/44 on hidden duplicates. The sample-to-population reconciliation (451/457 rows matched to engine v3) and the distinct-finding sensitivity analysis (0.613 vs 0.624) bound the effect of the v2→v3 frame

change. Corpus snapshots are pinned by commit SHA; the corpus is rebuildable from the released collectors and logs.

0.7 7. Related work

0.7.1 7.1 Empirical studies of Kubernetes misconfiguration

The closest predecessor is Rahman et al.'s empirical study of security misconfigurations in open-source Kubernetes manifests Rahman et al (2023). Mining 2,039 manifests from 92 repositories, they characterize 11 categories of security misconfiguration, build a detection tool (SLI-KUBE) to quantify their frequency (1,051 misconfigurations), and validate by reporting ten misconfigurations to practitioners, who agreed to fix 60%. We build on their characterization in four ways: an order-of-magnitude larger corpus with a second, curated arm (rendered Helm charts) that supports a curation comparison; a broader practice set (25 rules across security, resources, reliability, images, RBAC, and secrets, rather than security alone); separation of production manifests from examples and fixtures, which their manifest-level counting does not attempt; and a validation design that estimates *per-rule precision* over a stratified sample with a documented protocol and a blinded agreement check, rather than fix-agreement on a small set of reports. Their headline conclusion — insecure configurations are widespread — is corroborated by our data at larger scale; our contribution is establishing *which* of the counts can be believed, and by how much.

Zhang et al. take a defect-centered view of the same ecosystem Zhang et al (2026). Mining 2,260 Kubernetes configuration scripts from 185 open-source repositories, they extract 719 configuration defects and organize them into 15 categories, seven of which had not been reported in other software systems. Where our rules are seeded from hardening guidance, their categories are mined from defects that actually occurred, many of them functional defects outside any security taxonomy. Their tool analysis asks which of the 15 categories eight publicly available static analysis tools *can* detect (eight categories), reporting precision and recall against the mined defect set — highest for data-field defects; for the two categories no tool detects, they build a linter and find 26 previously unknown defects, confirmed by practitioners, 19 of them already fixed. The difference in ground truth is the difference in question: category-level tool support measured against historical defects cannot say whether a tool's findings on a given corpus are correct. For RQ3 we therefore score four tools' actual outputs on identical input against protocol reference labels — detection on reference-positive findings, flagging on reference-benign findings, and the counting semantics that make raw tool counts diverge — the evidence a practitioner choosing a scanner, or trusting one's output, actually consumes.

Minna et al. mine Helm charts from Artifact Hub and compare seven analyzer tools by their default policies, then evaluate whether LLMs can mitigate the reported misconfigurations Minna et al (2025) — in this journal. Their tool comparison aggregates common and unique findings per tool and includes a manual false-positive check on a subset. Our study asks the prior question their pipeline assumes away: whether the tools' reports on natural input are *correct*, rule by rule, against the reference labels — and finds that for context-sensitive rules they frequently are not (Section 4.3), a

result that also bears on their mitigation loop, since an LLM asked to fix a false positive can only damage a working configuration. The studies are complementary: they evaluate repair; we evaluate the measurement the repair depends on. We also add the raw-manifest arm and the production/fixture separation their chart-only corpus does not require.

A third recent line works from developer reports rather than manifests: Ghorab et al. derive a taxonomy of Kubernetes security misconfigurations from 2,662 Stack Overflow issues, trace how the issues evolve with project maturity, and evaluate LLM repair, reaching 98.5% correction accuracy with a schema-guided validation framework Ghorab et al (2026). Their unit of analysis — what developers *report struggling with* — complements ours: what production-shaped code *contains*, and whether automated findings about it are valid. Their repair results, like Minna et al.'s, presuppose that reported misconfigurations are genuine, which is the premise our validation tests.

Bufalino et al. study a single misconfiguration family in depth Bufalino et al (2025): network configurations that enable lateral movement inside a cluster. Evaluating 287 open-source applications from six organizations with their own analysis, they identify 634 network misconfigurations — beyond what state-of-the-art scanners report — and close a loop our study does not attempt: responsible disclosure to the organizations, severity assessed together with them, and fixes in more than thirty applications. Their study sits at the practitioner-impact end of the spectrum — one family, deployed applications, remediation — where ours sits at the measurement end: 25 practices across the published manifest ecosystem, with the validity of the counts established before anyone acts on them. The two are complementary, and their disclosure results show what validated prevalence findings can set in motion.

Beyond Kubernetes, configuration errors have been studied empirically in commercial and open-source systems Yin et al (2011), and the systems approaches to tackling them surveyed in Xu and Zhou (2015); studies of infrastructure-as-code ecosystems (Dockerfiles, Puppet, Terraform, CI configuration) generally find insecure defaults and smell-like patterns to be widespread. Those results motivate ours; the Kubernetes setting differs in that the configuration surface is a single, structured API with platform-supplied defaults for nearly every field, which is what makes a uniform 25-rule measurement across two distribution channels possible.

0.7.2 7.2 Scanner evaluation and benchmarks

Eom et al. present the first systematic benchmark of Kubernetes misconfiguration scanners Eom et al (2026): ten tools, whose documented coverage they found too ambiguous to take at face value, so they reverse-engineered precise detection targets from implementations and dynamic testing, then measured recall on a constructed benchmark of 4,109 misconfiguration files (281 unique misconfigurations), finding best recall of 43.8% (static) and 62.3% (dynamic). Their ground truth is *constructed* — each benchmark file is a known misconfiguration — which is the right design for measuring recall, and the wrong one for measuring what happens on real manifests, where the open question is not "does the tool fire on a seeded defect" but "is this firing, on this real object, a genuine misconfiguration." That is the quantity we measure: per-rule precision of findings on natural corpora, against protocol-based reference labels,

plus prevalence — neither of which a seeded benchmark can report. The two results also converge from opposite directions: their finding that scanners adopt different criteria for the same documented misconfiguration is the implementation-level explanation of our measurement-level finding that identical input yields ˜1.4× count spreads through counting semantics (Section 4.4). Where they audit ten scanners against a specification, we audit four against the wild.

Industry comparisons of hardening guidelines and scanner scoring (e.g., the Dynatrace Research comparison of eight guidelines and ten scanners) likewise report substantial coverage and scoring disparities; they share our conclusion that scanner outputs are not mutually comparable, reached there by guideline mapping rather than by validated findings.

0.7.3 7.3 Benchmarks and hardening guidance

The CIS Kubernetes Benchmark and the NSA/CISA Kubernetes Hardening Guide define the practice standard our rules operationalize, and the Pod Security Standards define the platform's own posture tiers. Taxonomy v2 (Section 4.3) can be read as an empirical annotation of those documents: which of their recommendations correspond to states that are wrong in essentially every context (Tier 1), which to hardening opportunities (Tier 2), and which to hygiene (Tier 3) — and, through the five false-positive context classes, where the guidance needs contextual exceptions that the documents themselves do not state (node-infrastructure agents being the largest).

0.7.4 7.4 Positioning

Dimension	Rahman et al. 2023	Minna et al. 2025	Eom et al. 2026	Zhang et al. 2026	Bufalino et al. 2025	This study
Corpus	2,039 manifests, 92 repos	Artifact Hub charts (all)	4,109 constructed files	2,260 scripts, 185 repos	287 deployed applications, 6 organizations	16,243 docs, 2 arms (60 repos + 151 charts)
Practices	11 security categories	tools' default policies	281 misconfigurations (union of 10 tools)	15 defect categories (all defect types)	network misconfigurations (lateral movement)	25 rules, 6 families
Ground truth	practitioner fix-agreement (10 reports)	manual FP check on a subset	constructed by design	719 defects mined from project history	own analysis; severity assessed with the organizations on disclosure	457-finding stratified sample, blinded subset, protocol + adjudication, author-adjudicated
Tool treatment	builds one tool	7 tools, common/unique counts + LLM repair	10 tools, recall on benchmark	8 tools: category-level detection support; precision/recall vs mined defects	own detector compared against state-of-the-art scanners	4 tools on identical natural input: coverage, count semantics, accuracy vs reference labels

Dimension	Rahman et al. 2023	Minna et al. 2025	Eom et al. 2026	Zhang et al. 2026	Bufalino et al. 2025	This study
Production vs fixture	not separated	n/a (charts)	n/a (constructed)	n/a (defect mining)	n/a (deployed applications)	separated; content-over-tag
Prevalence	manifest-level counts	chart-level counts	none (benchmark)	defect counts by category	634 misconfigurations across the 287 applications	rule workload-level, cluster-level, precision-adjusted

In one sentence: prior work establishes that Kubernetes misconfiguration exists (Rahman et al.), recurs as defects in project histories (Zhang et al.), and is exploitable in deployed applications (Bufalino et al.); that tools report it inconsistently and LLMs repair it imperfectly (Minna et al.); and that scanners miss much of what they claim to cover on seeded cases (Eom et al.). This study establishes how much of it there is in production-shaped code, how much of what any rule reports is real, and why the tools' numbers cannot be compared naively — the measurement foundation these directions implicitly rely on.

0.8 8. Conclusion

Across 16,243 real-world documents, Kubernetes misconfiguration is not a story of reckless operators but of permissive defaults accepted at scale: tokens automounted in ˜9 of 10 production workloads, limits and digest pinning absent as the norm, and the explicitly dangerous settings comparatively rare. The measurement itself needs care — a third of naive findings are benign in context, and the scanners the ecosystem relies on cover at most two-thirds of observed practices, agree mainly where danger is explicit, and diverge by bookkeeping where it is not. Both conclusions point the same way: harden the defaults, and label the measurements. These claims describe public, production-shaped manifests and rendered charts — the configuration the ecosystem publishes — not the private state of running clusters, which only their operators can observe.

0.9 Data availability and replication package

All artifacts are public: corpus collectors and per-source logs, the 25-rule engine, per-arm and per-tag statistics, the RQ3 coverage matrix, the complete labeling protocol (all versions and the 2026-10-27 addendum), the 457-row validation sample with evidence packets and final labels, and the analysis scripts, at https://github.com/mallyagirish/k8s-misconfig-study (Apache-2.0). GitHub sources are pinned by commit SHA (`results/github_commit_shas.json`) and Helm charts by name and version (`results/helm_render_log.json`), so the corpus is reconstructible as collected. An archival copy is deposited on Zenodo (DOI: 10.5281/zenodo.23966670). The corpus manifests themselves remain under their original licenses and are rebuilt, not redistributed.

Statements and Declarations

Funding

No funding was received for conducting this study.

Competing interests

The author declares no competing interests.

Ethics approval and consent to participate

Not applicable. The study analyzes publicly available open-source artifacts and involves no human subjects.

Author contributions

The author is the sole contributor to this work.

Use of artificial intelligence tools

AI tools were used in two ways, both disclosed in the methods. First, an AI assistant produced initial pre-labels for the validation sample under labeling protocol v2; every label was subsequently verified or adjudicated by the author under the governing protocol v3, and agreement was measured on a blinded subset labeled by the author before seeing the pre-labels (Section 3.4). Second, AI assistance was used for software development support and language editing during preparation of this manuscript. The author reviewed all output and takes full responsibility for the content.

References

Bufalino J, Martin-Navarro JL, Di Francesco M, et al (2025) Inside job: Defending Kubernetes clusters against network misconfigurations. In: Proceedings of the ACM on Networking, https://doi.org/10.1145/3749220

Center for Internet Security (2026) CIS Kubernetes Benchmark. Version current at time of study (accessed October 2026)

Eom H, Suk B, Hwang S (2026) An empirical study and benchmark of Kubernetes misconfiguration scanners. In: Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), https://doi.org/10.1145/3832297

Ghorab MA, Abdel Latif A, Saied MA (2026) Kubernetes misconfigurations in the wild: Taxonomy, evolution, and automated repair with large language models. ArXiv:2609.27030

kube-linter, kubeaudit, Polaris, and Checkov projects (2026) Tool documentation and check catalogues. Versions as recorded in the replication package

Kubernetes Project (2026) Pod Security Standards and security context documentation. Kubernetes.io/docs (accessed October 2026)

Minna F, Massacci F, Tuma K (2025) Analyzing and mitigating (with LLMs) the security misconfigurations of Helm charts from Artifact Hub. Empirical Software Engineering 30:132. https://doi.org/10.1007/s10664-025-10688-0

National Security Agency, Cybersecurity and Infrastructure Security Agency (2022) Kubernetes Hardening Guide

Rahman A, Shamim SI, Bose DB, et al (2023) Security misconfigurations in open source Kubernetes manifests: An empirical study. ACM Transactions on Software Engineering and Methodology 32(4):1–36. https://doi.org/10.1145/3579639

Xu T, Zhou Y (2015) Systems approaches to tackling configuration errors: A survey. ACM Computing Surveys 47(4):70:1–70:41. https://doi.org/10.1145/2791577

Yin Z, Ma X, Zheng J, et al (2011) An empirical study on configuration errors in commercial and open source systems. In: Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP), pp 159–172

Zhang Y, Paul U, d'Amorim M, et al (2026) Configuration defects in Kubernetes. IEEE Transactions on Software Engineering https://doi.org/10.1109/TSE.2026.3659785